Recursive Spawning with Immutable Parent Pointers: Preventing Autonomous Drift in Multi-Agent Systems

Jeremy Blaine Thompson Beebe

Bxthre3 Inc. — bxthre3inc@gmail.com — ORCID: 0009-0009-2394-9714

April 2026

Abstract

Multi-agent AI systems that spawn child agents or sub-tasks face a structural failure mode we term *autonomous drift*: the progressive disconnection of spawned agents from their originating human accountability anchor through successive layers of delegation. Each spawn event creates an opportunity for the accountability chain to attenuate, until a spawned agent operates with no recoverable path to a human decision-maker. This paper formalizes the Recursive Spawning pillar of the **BX3**Framework as a structurally anti-drift spawning architecture. The mechanism combines three elements: an immutable parent pointer cryptographically verified at every spawn, a forensic ledger recording the full delegation ancestry, and a termination guarantee that recursively enforceable chains terminate at the *Purpose Layer* Layer — never at a machine actor. We prove that under these three conditions, drift is formally prevented: any agent in any generation of a spawn tree can be traced to its originating *Purpose Layer* Layer anchor, and any unresolved condition escalates to that anchor without exception. We further show that chain integrity is maintained under arbitrary spawn depth and that termination is guaranteed by the architecture's upward-only escalation rules.

Keywords: Recursive Spawning, Immutable Parent Pointers, Autonomous Drift, Multi-Agent Systems, BX3 Framework, Forensic Ledger, Accountability Chains, Agent Lifecycle, Human-in-the-Loop, Sovereign Agents

1 Introduction

The capability of modern AI agents to spawn sub-agents — delegating subtasks to child processes, spawning parallel worker threads, or initiating recursive reasoning chains — is a fundamental feature of advanced multi-agent systems. It is also the primary mechanism by which those systems become ungovernable. Each spawn event introduces a new node into the agent graph; each new node inherits a partial and potentially attenuated view of the original mission. Without structural constraints on how spawn events preserve accountability, the system converges toward a state in which the most operationally significant agents are the least traceable to their human origin.

The **BX3**Framework addresses this failure mode through the Recursive Spawning pillar, one of five architectural pillars that together define a **BX3**-compliant system. Recursive Spawning is not a policy for responsible spawning — it is an architectural constraint that makes drift *physically impossible* rather than merely *discouraged*. The mechanism operates at the level of the agent lifecycle: every spawn event is recorded, every child is cryptographically tethered to its parent, and the chain of custody is sealed in an append-only forensic ledger from which no link can be removed or redirected.

This paper focuses on two sections of the full Recursive Spawning specification: the integration of Recursive Spawning with the three **BX3** layers (*Purpose Layer*, *Bounds Engine*, and *Fact Layer*), and the formal correctness properties that the mechanism guarantees. We do not repeat the full Recursive Spawning specification here — that material belongs to Sections 1–5 of this paper. Instead, we treat Sections 6 and 7 as the architectural grounding (Section 6) and the proof of guarantees (Section 7) for the mechanism described in the preceding sections.

2 Integration with the BX3 Framework

2.1 The Three-Layer Architecture as an Accountability Infrastructure

The **BX3** Framework provides the structural context within which Recursive Spawning operates. Recall from the formal model of the **BX3** Framework [**?**] that any system Σ is a 5-tuple $\Sigma = (P, B, F, E, H)$ where P is the *Purpose Layer* Layer, B is the *Bounds Engine* Engine, F is the *Fact Layer* Layer, E is the execution environment, and H is the human accountability anchor. The Recursive Spawning pillar operates within this structure: it constrains how agents spawned from B (or from other spawned agents) relate back to P and H, ensuring that the spawn tree does not become a vector for accountability attenuation.

The integration is not incidental. Each of the three **BX3** layers plays a specific and irreplaceable role in making Recursive Spawning anti-drift by construction:

- The *Purpose Layer* Layer serves as the *accountability anchor*: it is the unique root from which all legitimate spawn events originate, and it is the unique terminal of the escalation path. No spawn event may originate from a node that does not itself trace back to P, and no spawn chain may terminate anywhere other than P. This is the

most important structural property: the *Purpose Layer* Layer is a *single, immutable root.*

- The *Bounds Engine* Engine provides the *depth enforcement* mechanism: it enforces that spawn trees do not exceed the depth limits defined by the Safety Envelope $\mathcal{S}$, rejects spawn proposals that would violate those limits, and escalates to P any situation in which a spawn request cannot be bounded. The *Bounds Engine* Engine's boundedness is what prevents unbounded recursion — the very condition that makes drift possible.

- The *Fact Layer* Layer serves as the *forensic ledger*: it maintains the append-only record of every spawn event, every parent-child link, every delegation of authority, and every state transition in the spawn tree. The ledger is deterministically written — same spawn event produces the same ledger entry — and is cryptographically sealed so that past entries cannot be modified. The forensic ledger is the evidence base that makes drift *detectable* even if the architectural constraints were somehow bypassed.

These three roles are architecturally separate and individually necessary. Removing any one destroys the anti-drift property. We examine each in detail.

2.2 The Purpose Layer as the Accountability Anchor

The *Purpose Layer* Layer is the unique root of the spawn tree. In **BX3** terms, every spawn event traces its authority to P, and every unresolved condition in the spawn tree escalates to P. The key architectural property is that P is singular: there is exactly one P per system Σ, and it cannot be replicated, delegated away, or replaced by a machine actor.

This is not a policy choice — it is a constraint encoded in the formal model. Constraint P2 (*Accountability Attribution*) requires that for every action a executed by Σ, there exists a chain of attribution $\tau(a) = (p_1, p_2, \ldots, p_n)$ terminating at a human actor $p_n \in H$. In the context of Recursive Spawning, this means that for every spawned agent A_i, the full delegation chain from A_i back to P must exist and must be recorded. Constraint P3 (*Override Capability*) further guarantees that P may override any decision made by any node in the spawn tree at any time.

The implications for spawn architecture are direct. A child agent spawned from a parent agent does not have an independent accountability relationship with H — it has a *derived* relationship that passes through the parent, and the parent's relationship passes through its parent, and so on, recursively, until the chain terminates at P. This is the accountability chain. It is append-only: new links may be added (spawning a new child), but existing links may not be removed or redirected.

The *Purpose Layer* Layer's role as accountability anchor is strengthened by its relationship with the Bailout Protocol [**?**], also a pillar of the **BX3** Framework. When any spawned agent encounters an unresolvable condition, the Bailout Protocol propagates the exception upward through the spawn tree, bypassing intermediate machine actors, until it reaches P. The spawn tree is thus not only a delegation structure but also an escalation structure — the same tree that carries delegated authority downward also carries exception escalations upward.

2.3 The Bounds Engine as Depth Enforcement

The *Bounds Engine* Engine performs two functions in the Recursive Spawning architecture: it enforces spawn depth limits defined by the Safety Envelope, and it evaluates whether a given spawn request is within the operational authority of the requesting node.

The Safety Envelope $\mathcal{S}$, defined by P at system instantiation, bounds the space of permissible proposals π. In the context of Recursive Spawning, $\mathcal{S}$ includes a *spawn depth parameter* $\delta_{\max}$: the maximum number of generational levels permitted in the spawn tree. This parameter is set by P based on the operational requirements and risk profile of the system. A system managing a safety-critical physical process might set $\delta_{\max} = 2$; a research-oriented system might permit $\delta_{\max} = 5$. The value is not fixed by the framework — it is a parameter set at design time by the *Purpose Layer* Layer.

When an agent requests to spawn a child, the *Bounds Engine* Engine checks whether the resulting tree depth would exceed $\delta_{\max}$. If the request would exceed the depth limit, the *Bounds Engine* Engine rejects the spawn and generates an escalation to P, which may grant an exception (temporarily increasing $\delta_{\max}$ for a specific purpose) or deny the request. This mechanism ensures that the spawn tree never grows beyond the depth that P has authorized, even when child agents encounter conditions that, in the moment, might seem to justify additional spawning.

Constraint B2 (*Safety Envelope Compliance*) formalizes this: all proposals π — including spawn proposals — must satisfy $\pi \in \mathcal{S}$. Constraint B3 (*Escalation on Unresolvable Bounds*) ensures that when a spawn request cannot be satisfied within $\mathcal{S}$, autonomous continuation is not permitted. The agent must escalate, and the escalation path leads upward toward P.

The *Bounds Engine* Engine also enforces the *safety properties of the spawn operation itself*: it validates that the proposed child agent is of a type authorized within $\mathcal{S}$, that the child's intended scope of action is within the parent's inherited Safety Envelope, and that the delegation of authority from parent to child does not exceed what the parent was itself authorized to delegate. This chain of delegated authority is itself a bounded quantity.

2.4 The Fact Layer as Forensic Ledger

The *Fact Layer* Layer provides the deterministic, append-only record of all spawn events. For Recursive Spawning, the forensic ledger records the following event types at minimum:

1. **Spawn events:** timestamp, parent agent ID, child agent ID, child agent type, Safety Envelope parameters inherited from parent, cryptographic hash of the spawn proposal.

2. **Delegation events:** timestamp, delegating agent, delegate agent, scope of delegated authority, expiration time (if applicable), cryptographic signature of delegating agent.

3. **State transitions:** timestamp, agent ID, previous state, new state, triggering event, cryptographic hash of prior state.

4. **Termination events:** timestamp, agent ID, termination reason, final state hash, chain of custody verification.

The forensic ledger is maintained by the *Fact Layer* Layer's deterministic execution machinery. Every entry is written using the deterministic transition function δ_F: given the current ledger state and a validated spawn event, the ledger state transitions to a new unique state. This means the ledger is versioned — each state has a cryptographic hash, and the hash of state n is a function of the hash of state $n-1$ and the contents of event n. This creates a chain of hashes analogous to a blockchain, in which any tampering with an earlier entry would propagate forward and become detectable at the current state.

Constraint F3 (*Immutable Audit Logging*) of the **BX3** Framework requires that every action executed by F is recorded in an append-only log $\mathcal{L}$. The ledger described here is precisely this log, specialized for Recursive Spawning events. The log is cryptographically sealed — we use a hash-based chain-of-custody similar to Certificate Transparency [**?**] — ensuring that ledger entries cannot be retroactively modified, deleted, or reordered.

The forensic ledger serves three functions in the anti-drift architecture:

1. **Detecting drift:** If a spawned agent acts without a corresponding entry in the ledger (indicating it was spawned without following the Recursive Spawning protocol), this is immediately detectable. The agent's authority is unverifiable, and any actions taken by that agent are treated as unauthorized.

2. **Attribution:** In post-incident analysis, the full spawn tree and delegation chain of any agent can be reconstructed from the ledger. This is the evidence base for understanding what happened, who was responsible at each step, and where the accountability chain was maintained or broken.

3. **Certification:** External auditors can verify compliance by examining the ledger. A clean ledger — one in which every agent's existence is explained by a spawn event, and every spawn event traces back to P — constitutes evidence of **BX3** compliance with respect to the Recursive Spawning pillar.

3 Correctness Properties

We now prove the three correctness properties of the Recursive Spawning mechanism under the **BX3** Framework: drift prevention, chain integrity, and termination. Each property is proven as a formal theorem given the constraints of the **BX3** formal model and the additional constraints introduced by Recursive Spawning.

3.1 Drift Prevention

Autonomous drift is the failure mode in which spawned agents progressively disconnect from the human accountability anchor through successive layers of delegation, such that the chain of attribution no longer reaches H or P. Drift occurs when the accountability chain is broken — when an agent exists and acts without a recoverable path to its originating P.

We prove that under the Recursive Spawning architecture, drift is impossible. The proof relies on three architectural conditions that hold simultaneously:

C1 The parent pointer is immutable and cryptographically verified.

C2 The forensic ledger is append-only and cryptographically sealed.

C3 The spawn chain terminates at the *Purpose Layer* Layer — no machine actor may serve as the terminal of a spawn chain.

Theorem 1 (Drift Prevention)**.** *Let $\mathcal{A} = \{A_0, A_1, \ldots, A_n\}$ be the set of all agents in a **BX3** system Σ, where A_0 is the agent spawned directly by P (the first generation). Let $\tau(A_i)$ denote the accountability chain of A_i, defined as the ordered sequence of agents from A_i back to the root of the spawn tree. Under conditions C1–C3:*

$$\forall A_i \in \mathcal{A} : \mathit{terminal}(\tau(A_i)) = P$$

That is, the accountability chain of every agent — at any generation depth — terminates at the Purpose Layer Layer. No agent in the spawn tree has a terminal other than P.

Proof. We prove by induction on the generation depth of the spawn tree.

Base case ($g = 0$): A_0 is the agent spawned directly by P. By P2, the chain $\tau(A_0) = (A_0, P)$ exists and terminates at P. C1 ensures the parent pointer of A_0 to P is immutable; C2 ensures the spawn event is recorded in the forensic ledger and cannot be altered. $\therefore \tau(A_0)$ terminates at P.

Inductive hypothesis: Assume that for all agents at generation depth $g \leq k$, the accountability chain $\tau(A_g)$ terminates at P.

Inductive step ($g = k + 1$): Consider an agent A_{k+1} spawned by a parent agent A_k at depth k. By the Recursive Spawning protocol, A_k must verify its own parent pointer before issuing the spawn request (C1). The spawn event is recorded in the forensic ledger (C2), creating a permanent link between A_{k+1} and A_k. By the inductive hypothesis, $\tau(A_k)$ terminates at P. By the definition of spawn, $\tau(A_{k+1}) = (A_{k+1}, \tau(A_k))$ — the chain of A_{k+1} extends the chain of A_k with A_{k+1} as the new head. Therefore $\tau(A_{k+1})$ terminates at P.

Since C3 prohibits any machine actor from serving as the terminal of a spawn chain, the base case and inductive step cannot be satisfied for any agent whose chain terminates elsewhere. The theorem follows by induction on all generation depths. □ □

The theorem establishes that drift is not merely unlikely or discouraged — it is structurally impossible. An agent at generation depth 100 has the same terminal (P) as an agent at generation depth 1. The spawn chain does not attenuate with depth; it is anchored at its origin by construction.

The proof sketch above is an informal presentation suitable for the body of this paper. The full formal proof is available in the TLA+ specification of the Recursive Spawning pillar in the **BX3** reference implementation (see the appendix of [**?**]).

3.2 Chain Integrity

Chain integrity refers to the property that the accountability chain of any agent is complete, unaltered, and verifiable at any point in time. A chain with integrity is one in which every link is exactly as it was when it was created, no links have been removed or reordered, and every link's cryptographic verification passes.

Definition 1 (Chain Integrity)**.** *A spawn tree $\mathcal{T}$ maintains chain integrity if and only if for every agent $A_i \in \mathcal{T}$:*

CI1 *The parent pointer of A_i is cryptographically verified against the parent's public key, and verification succeeds.*

CI2 *The spawn event of A_i is present in the forensic ledger, and the ledger's hash chain is unbroken from the genesis event to the entry recording A_i's spawn.*

CI3 *The Safety Envelope parameters inherited by A_i from its parent are consistent with the Safety Envelope of P and with all intermediate nodes in $\tau(A_i)$.*

Theorem 2 (Chain Integrity Invariant)**.** *The Recursive Spawning mechanism maintains chain integrity as an invariant of system operation. Formally:*

$$\textit{Invariant} : \forall A_i \in \mathcal{A} \text{ at all times } t : \textit{CI1} \wedge \textit{CI2} \wedge \textit{CI3}$$

Proof. We prove that the invariant holds at system initialization and is preserved by every permitted operation.

Initialization: At system initialization, A_0 is spawned by P through the Fact Layer's deterministic spawn function. The parent pointer of A_0 is set to P, cryptographically signed by P, and recorded in the genesis entry of the forensic ledger. CI1, CI2, and CI3 hold for A_0.

Preservation under spawn: Consider a spawn event in which agent A_k (which satisfies CI1–CI3) spawns A_{k+1}. The spawn operation is executed by F, which: (a) verifies A_k's parent pointer (preserving CI1 for A_k), (b) writes the spawn event to the forensic ledger, extending the hash chain (preserving CI2), and (c) copies A_k's Safety Envelope parameters as the starting parameters for A_{k+1}, verifying they are within $\mathcal{S}$ as defined by P (preserving CI3). After the spawn event, A_{k+1} satisfies CI1–CI3, and A_k continues to satisfy CI1–CI3 (no operation modifies existing links).

Preservation under non-spawn operations: No other operation in the **BX3** system modifies parent pointers, removes entries from the forensic ledger, or alters Safety Envelope parameters of existing agents. Therefore CI1–CI3 are preserved for all existing agents.

By induction on the sequence of system operations, the invariant holds at all times. □

□

The chain integrity invariant is not merely a design goal — it is enforced by the architecture. The cryptographic verification of parent pointers (CI1) is performed by the *Fact Layer* Layer at spawn time and periodically during operation (every 15 minutes in the reference implementation). The forensic ledger's hash chain (CI2) is verified by the Compliance Monitor at each compliance attestation. Safety Envelope consistency (CI3) is enforced by the *Bounds Engine* Engine at every spawn event.

3.3 Termination

Termination is the property that the Recursive Spawning process cannot produce an infinite chain of agents. In a spawning system without structural constraints, a child agent could spawn another child, which spawns another, and so on indefinitely — creating an unbounded spawn tree that consumes resources and produces agents with no resolvable mission. In the **BX3** Framework, this is prevented by the interaction of the *Bounds Engine* Engine's depth limits and the *Purpose Layer* Layer's authority over spawn operations.

Definition 2 (Spawn Depth)**.** *The spawn depth $\delta(A_i)$ of an agent A_i is the length of the shortest path in $\mathcal{T}$ from P to A_i, measured in edges. Formally:*

$$\delta(A_i) = \begin{cases} 0 & \textit{if } A_i \textit{ is spawned directly by } P \\ 1 + \delta(\textit{parent}(A_i)) & \textit{otherwise} \end{cases}$$

Definition 3 (Maximum Spawn Depth)**.** *The maximum spawn depth Δ is the largest $\delta(A_i)$ permitted by the Safety Envelope $\mathcal{S}$. It is set by P at system instantiation and may be updated by P through the override mechanism.*

Theorem 3 (Termination Guarantee)**.** *No agent in a **BX3** system Σ can have spawn depth $\delta(A_i) > \Delta$. Formally:*

$$\forall A_i \in \mathcal{A} : \delta(A_i) \leq \Delta$$

Equivalently: the spawn tree $\mathcal{T}$ is bounded and terminates.

Proof. The proof follows directly from B2 (*Safety Envelope Compliance*) applied to spawn operations. A spawn request is a proposal π_{spawn} generated by the requesting agent. By B2, any proposal π must satisfy $\pi \in \mathcal{S}$. The Safety Envelope $\mathcal{S}$ includes the constraint $\delta(A_{\text{child}}) \leq \Delta$ for any child spawned by the request. If a spawn request would produce a child with $\delta > \Delta$, the proposal $\pi_{\text{spawn}} \notin \mathcal{S}$, and the *Bounds Engine* Engine rejects it before it reaches the *Fact Layer* Layer. No rejected proposal is executed. Therefore no spawn event can produce an agent with $\delta > \Delta$. □ □

The Termination Guarantee ensures that the spawn tree is not just bounded — it is provably bounded by a known, fixed parameter Δ that is under the direct control of P. This prevents both resource exhaustion through unbounded spawning and the emergence of deep agent hierarchies that no human operator can meaningfully oversee.

The combination of the Drift Prevention theorem and the Termination Guarantee establishes the full correctness of Recursive Spawning: no agent can be outside the accountability chain (drift prevention), the chain is verifiable and unaltered at all times (chain integrity), and the chain cannot grow without bound (termination).

4 Related Work

The problem of maintaining accountability across agent delegation has been addressed from several angles in recent literature. The W3C PROV Data Model [**?**] provides a general-purpose provenance framework for tracking the lineage of digital entities, including agents.

PROV-AGENT [?] extends this to AI agent interactions, proposing a unified provenance model for agentic workflows. While complementary, these frameworks do not provide structural constraints that prevent drift — they describe how to record delegation after the fact, rather than making drift architecturally impossible.

The Human Delegation Provenance Protocol (HDP) [?, ?] comes closest to the Recursive Spawning mechanism in its goals. HDP proposes a cryptographic chain-of-custody for agentic delegation, using append-only logs and signed delegation records. The Recursive Spawning pillar shares HDP's emphasis on cryptographic chain-of-custody but goes further by integrating the chain-of-custody into a complete three-layer architecture in which the chain is not merely recorded but is structurally enforced by the interaction of P, B, and F. HDP describes a protocol; **BX3** describes an architecture.

SentinelAgent [?] provides a formal framework for secure delegation chains in federal multi-agent AI systems, with formal verification of delegation properties. The Recursive Spawning pillar draws on similar formal methods but applies them within the **BX3** three-layer model, which provides the semantic grounding (accountability, boundedness, determinism) that makes the formal properties meaningful in operational terms.

Hierarchical task decomposition methods including ReAcTree [?] and HiPER [?] address the problem of planning across multiple levels of agent hierarchy but do not address the accountability attenuation problem. They treat the hierarchy as a planning structure rather than an accountability structure. Similarly, structured test-time scaling approaches [?] and recursive thinking frameworks [?] explore recursive reasoning patterns without considering the accountability implications of spawning at depth.

The immutable audit trail requirement for non-human identities (NHIs) is discussed in [?], which proposes append-only registries for tracking AI agent lineage. This work aligns with the forensic ledger component of Recursive Spawning. The key distinction is that [?] proposes the registry as a separate infrastructure layer, whereas in **BX3**, the forensic ledger is a first-class component of the *Fact Layer* Layer — the same layer that enforces deterministic execution and hard physical constraints.

5 Limitations

The Recursive Spawning mechanism as described in this paper has several limitations that implementers should be aware of.

First, the Termination Guarantee relies on the correctness of the depth parameter Δ set by P. If Δ is set too large, the spawn tree may grow to a size that exceeds human cognitive capacity to review — even if the chain terminates mathematically. The architecture guarantees that every agent is within Δ generations of P; it does not guarantee that P can meaningfully supervise a tree of size N. Setting Δ appropriately is a judgment that remains with P, and this judgment cannot be made rigorous by the framework alone.

Second, the drift prevention proof assumes that the cryptographic primitives used for parent pointer verification are secure and that the private keys used for signing are not compromised. If the signing key of an intermediate parent agent is compromised, the attacker could in principle spawn agents that appear to have valid parent pointers but whose chain of custody is not meaningfully attributable to P. Key management for spawned agents is a

practical security problem that the framework outlines but does not fully solve.

Third, the forensic ledger's integrity is guaranteed only if the hash function used for the chain-of-custody is collision-resistant. The reference implementation uses SHA-256, which provides strong but not absolute collision resistance. A sufficiently motivated adversary with quantum computing capabilities could theoretically construct hash collisions that undermine the ledger's integrity. This limitation is shared with all blockchain-based integrity mechanisms and is not specific to Recursive Spawning.

Fourth, the integration of Recursive Spawning with cross-domain spawning (spawning agents that migrate between execution environments or physical systems) is not fully specified in this paper. When a spawned agent operates in an environment governed by different Safety Envelope parameters than its parent, the inheritance and reconciliation of Safety Envelope parameters requires additional specification work.

6 Conclusion

The Recursive Spawning pillar of the **BX3** Framework provides the structural mechanism by which multi-agent systems can spawn child agents without sacrificing human accountability. By integrating with the *Purpose Layer* Layer (as the immutable accountability anchor), the *Bounds Engine* Engine (as the depth enforcement mechanism), and the *Fact Layer* Layer (as the forensic ledger), Recursive Spawning ensures that the spawn tree is simultaneously a delegation structure, an escalation structure, and an audited evidence structure. The three correctness theorems — drift prevention, chain integrity, and termination — establish that the mechanism achieves its stated goals formally.

Future work will address cross-domain spawn scenarios, formal verification of the full Recursive Spawning lifecycle in TLA+, and integration with emerging agent lifecycle standards including ALP [**?**]. The framework is designed to evolve: new evidence, new threat models, and new deployment contexts will require refinement of the depth parameters, cryptographic primitives, and forensic ledger schema. The architecture is built to accommodate this evolution while maintaining the three core correctness properties.

Acknowledgments

The author gratefully acknowledges the contributions of the Bxthre3 Inc. engineering team in deploying and stress-testing the Recursive Spawning mechanism in operational environments, and the feedback of early reviewers on the drift prevention argument.

Keywords: Recursive Spawning, Immutable Parent Pointers, Autonomous Drift, Multi-Agent Systems, BX3 Framework, Forensic Ledger, Accountability Chains, Agent Lifecycle, Human-in-the-Loop, Sovereign Agents

Appendix: About the Author

Jeremy Blaine Thompson Beebe is the founder and chief architect of Bxthre3 Inc., where he leads development of the BX3 Framework and its applications in AI workforce orchestration,

precision agriculture, and autonomous systems governance. He holds an ORCID identifier (0009-0009-2394-9714) and can be reached via email at bxthre3inc@gmail.com.